

ThreadM ConfusionBuffer Anki

Thread M — Confusion Buffer & Anki Pack (Implement-by-Hand ML)

Companion to the Threads section (runs from the Phase-1 self-test in Block B). ~165 hrs — the highest-ROI depth-builder and the exact skill DeepMind-style coding rounds test (implement a loss/attention from a blank file).

How to use: one primitive from a **blank file, no autocomplete/AI**, in ~30–45 min; **gradient-check** every numeric implementation; revisit spaced; build up to a small end-to-end system. Cards here capture the *key subtlety* and the *numerical gotcha* for each primitive — the things that separate "I read it" from "I can code it cold." **Triangulation targets:** *softmax/logsumexp stability, the backprop δ -recursion, BatchNorm train-vs-eval, and the reparameterization trick.*

The implement-cold list (rotate through these)

backprop (MLP) · scalar autodiff engine · scaled-dot-product + multi-head attention · softmax / log-softmax · cross-entropy from logits · Dice/Focal/Tversky · Batch/LayerNorm · SGD/Adam · a DDPM step · PCA-via-SVD · k-means · logistic/linear regression · a message-passing GNN layer · top-k/nucleus sampler · positional encodings (sinusoidal/RoPE) · KV-cache · a small VQE circuit (Phase Q) · later: translate one to JAX.

Anki deck (Q → A)

Deck A · The primitives — key subtlety each

- Q:** Backprop through an MLP — the core recursion + what you must cache? → **A:** Cache each layer's pre-activation z and activation a on the forward pass; backward: $\delta_L = \partial L / \partial z_L$, then $\delta_{l-1} = (W_l^T \delta_L) \odot f'(z_{l-1})$; grads: $\partial L / \partial W_l = \delta_L a_{l-1}^T$, $\partial L / \partial b_l = \delta_L$.
- Q:** A scalar autodiff engine (micrograd) — the two must-haves? → **A:** Each op records its inputs + a local `_backward` closure; run backward in **reverse topological order, accumulating** (+) gradients (a node used twice sums both paths).
- Q:** Scaled-dot-product attention — the three subtleties? → **A:** Divide logits by $\sqrt{d_k}$; apply the mask by adding $-\infty$ (or a large negative) *before* softmax; use a numerically-stable softmax (subtract max).
- Q:** Multi-head attention — the reshape trick? → **A:** Project to $(B, n, h \cdot d_{\text{head}})$, reshape/transpose to $(B, h, n, d_{\text{head}})$, do batched attention, transpose back and concat, then a final linear.
- Q:** Softmax — the stability fix? → **A:** Subtract the row max before exponentiating: $\text{softmax}(x) = \text{softmax}(x - \max(x))$ → avoids overflow.
- Q:** log-softmax / logsumexp — why not $\log(\text{softmax})$? → **A:** Computing softmax then log loses precision and can $\log(0)$; use $\text{logsumexp} = m + \log \sum e^{x-m}$ directly.
- Q:** Cross-entropy from logits — the right way? → **A:** Combine with log-softmax: $\text{CE} = -(\text{logits}[\text{target}] - \text{logsumexp}(\text{logits}))$; never $\text{softmax} \rightarrow \text{log} \rightarrow \text{gather}$ (unstable). Gradient wrt logits = $\text{softmax} - \text{onehot}$.
- Q:** Dice loss — the numerical must? → **A:** Add ϵ to numerator and denominator $(2|P \cap G| + \epsilon) / (|P| + |G| + \epsilon)$ for stability on empty masks; use *soft* (probability) P for a differentiable loss.
- Q:** BatchNorm — what breaks if you get train/eval wrong? → **A:** Train uses batch mean/var (and updates running stats); **eval must use the running stats** — using batch stats at eval (or vice-versa) silently corrupts predictions.
- Q:** LayerNorm — what's normalized? → **A:** Mean/var over the *feature* dimension per token (not the batch), then scale/shift γ, β ; independent of batch size / sequence length.
- Q:** Adam — the parts people forget? → **A:** Bias-correction ($m = m / (1 - \beta_1^t)$, $v = v / (1 - \beta_2^t)$) and the ϵ inside the sqrt: $\theta \leftarrow \eta \cdot m / (\sqrt{v} + \epsilon)$; keep per-parameter state.
- Q:** A DDPM step — forward + loss? → **A:** Forward (closed form): $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$; train $\epsilon_\theta(x_t, t)$ with MSE to the sampled ϵ ; reverse subtracts predicted noise per the schedule.
- Q:** PCA via SVD — the pre-step? → **A:** **Center** the data (subtract the mean) first; then components = right-singular vectors; explained variance $\propto \sigma^2$.
- Q:** k-means — the two alternating steps + init? → **A:** Assign each point to the nearest centroid, then recompute centroids as cluster means; repeat; use k-means++ init to avoid poor local minima.
- Q:** A message-passing GNN layer — the aggregation op? → **A:** Gather neighbor GNN messages and **scatter-add** into each node (e.g., `index_add / segment-sum`), then apply the update MLP; keep it permutation-invariant.
- Q:** Top-k / nucleus sampling — the mechanics? → **A:** Scale logits by $1/\text{temperature}$ → softmax → for top-k keep the k largest; for nucleus keep the smallest set with cumulative prob $\geq p$ → renormalize → sample.
- Q:** Sinusoidal vs RoPE positional encoding — implement? → **A:** Sinusoidal: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$, cos for $2i+1$, added to embeddings. RoPE: rotate each (even, odd) dim pair of q/k by angle $\text{pos} \cdot \theta_i$ (a 2×2 rotation) → relative position in the dot product.
- Q:** KV-cache — what do you store and reuse? → **A:** Append each new token's K and V to per-layer buffers; attention for the new token reads the whole cache → avoids recomputing past K/V .

Deck B · Numerical-stability gotchas

- Q:** Two classic overflow/underflow bugs? → **A:** softmax without max-subtraction (overflow); log of a near-zero probability ($-\infty$) → use `logsumexp / clamp`.
- Q:** How do you gradient-check an implementation? → **A:** Compare the analytic gradient to a finite-difference estimate $(f(\theta+h) - f(\theta-h)) / 2h$ and check the **relative** error is $\sim 1e-6$ – $1e-4$.
- Q:** Why compute variance in fp32 / carefully? → **A:** The naive $(E[x^2] - E[x]^2)$ form catastrophically cancels; use a two-pass or Welford's algorithm, and do reductions in fp32 under mixed precision.
- Q:** Exploding/vanishing gradients — two implementation levers? → **A:** Proper init (He/Xavier) + gradient clipping (and residual/normalization); check gradient norms during training.
- Q:** Float equality trap? → **A:** Never test floats with `==`; compare within a tolerance.

Deck C · Frameworks & workflow

- Q:** PyTorch autograd essentials? → **A:** Tensors with `requires_grad` build a graph; `loss.backward()` fills `.grad`; wrap inference in

`torch.no_grad()` ; zero grads each step.

- **Q:** Why is einsum clarifying? → **A:** It names each axis and the contraction explicitly (e.g., `bhqd,bhkd->bhqk`), making attention/tensor ops unambiguous and less error-prone than reshape+matmul.
- **Q:** JAX in one line — the three transforms? → **A:** `jit` (compile), `grad` (autodiff), `vmap` (auto-batch) over **pure** functions; state (params) is passed explicitly.
- **Q:** JAX PRNG — why explicit keys? → **A:** JAX has no global RNG state; you split an explicit key for each random op → reproducible, parallelizable randomness.
- **Q:** Why implement from scratch at all? → **A:** It builds true mechanistic understanding *and* it's the literal interview task (implement a loss/attention/GNN from a blank file) — reading code ≠ being able to write it cold.

Deck D · Practice discipline (the thread)

- **Q:** The blank-file rule? → **A:** No autocomplete, no AI, ~30–45 min per primitive; if stuck >30 min, read the *mechanism* (not the code), close it, and rewrite from memory.
- **Q:** How to level up over the program? → **A:** Primitives → compose them into a small system (nanoGPT-scale, a mini-U-Net, a message-passing net) → then translate one primitive to JAX.
- **Q:** The checkpoint bar? → **A:** Implement any listed primitive cold, timed, gradient-checked, and explain every line.

Common misconceptions & traps

- **"Reimplementing is a waste — I'll just call the library."** It's the single highest-ROI depth-builder and the exact skill research-coding rounds test.
- **"Autograd means I never need to derive backprop."** You do — for interviews, for debugging vanishing gradients, and to reason about memory/compute.
- **"Vectorization is premature optimization."** In array code it's clarity + necessity; loops over tensors are the bug source, not the fix.
- **"A softmax is a softmax."** Without max-subtraction it overflows; without combining with log it's an unstable cross-entropy.
- **"BatchNorm just works."** The train/eval stats switch is a top production bug.

Glossary starter

backprop (δ-recursion, cached activations) · autodiff (topo order, grad accumulation) · scaled-dot-product / multi-head attention · softmax / logsumexp stability · cross-entropy-from-logits · Dice ε-smoothing · BatchNorm (running stats) / LayerNorm · Adam (bias correction) · DDPM step · PCA-via-SVD (centering) · k-means (+ +init) · message passing (scatter-add) · top-k/nucleus sampling · sinusoidal/RoPE PE · KV-cache · gradient checking · Welford variance · einsum · PyTorch autograd · JAX (`jit/grad/vmap`, PRNG keys) · Equinox.

Drills

Blank-file, timed, gradient-checked: micrograd (autodiff + MLP); multi-head attention (+ mask + $\sqrt{d_k}$); stable log-softmax + cross-entropy; Dice/Focal from scratch; BatchNorm + LayerNorm (train/eval); Adam; a DDPM forward+loss; PCA-via-SVD; k-means; a message-passing GNN layer; a top-k/nucleus sampler; RoPE. **Then:** rebuild one of these in JAX (`jit/grad/vmap`). **Compose:** a nanoGPT-scale model or a mini-U-Net from your own primitives.

Leaving bar (checkpoint)

Implement any primitive on the list from a blank file, timed, with a passing gradient check, and explain each line — plus at least one primitive re-expressed in JAX.